

Publicly Verifiable and Privacy-Preserving Score Voting for Decentralized Governance

Liang Zhang, *Member, IEEE*, Xingyu Wu, Haibin Kan*, *Member, IEEE*, Jiheng Zhang*

Abstract—In decentralized online governance, participants express preferences over proposals, candidates, budgets, or policy options, and the platform aggregates these preferences into collective decisions. Voting is a common operational interface for such preference aggregation, where each ballot encodes a user’s private preference vector. However, implementing privacy-preserving preference aggregation that ensures public verifiability and supports independently bounded multi-option score ballots in a decentralized environment presents significant challenges. Some systems rely on an administrator to handle key generation or tallying, which introduces a single point of failure. Other systems are limited to binary choices, require voters to remain online during tallying, or defer work over all ballots until finalization. To address these challenges, we define range-bound publicly verifiable secret sharing (RB-PVSS), which jointly binds encrypted polynomial shares, masked score commitments, and proofs that those scores lie in a public range. We construct RB-PVSS from SCRAPE PVSS and a modified constant-size range proof. One RB-PVSS instance packs scores for multiple options; valid instances are verified and aggregated online before talliers jointly reconstruct only the aggregate values. The protocol rejects malformed ballots, tolerates the stated bound of faulty talliers, and makes ballot and tallying messages publicly accountable. Finally, we analyze its security and evaluate its off-chain performance and on-chain scaling.

Index Terms—Preference aggregation, decentralized online governance, DAO, PVSS, voting, range proof

I. INTRODUCTION

Decentralized governance allows geographically distributed participants to make collective decisions about protocol upgrades, treasury allocations, public-goods funding, parameter changes, and the selection or prioritization of multiple proposals. Decentralized autonomous organizations (DAOs) [1]–[3] commonly implement these processes through blockchain smart contracts, which provide a publicly auditable execution environment without requiring a single organization to maintain the voting record. This transparency is valuable for accountability, but it also makes every submitted message per-

manently visible and therefore creates a fundamental tension between public verification and preference privacy.

This work considers a concrete preference-aggregation task for such governance settings. Let voter j assign a score $w_{jd} \in S = [a, b]$ to each of l governance options. The voter’s private ballot is $w_j = (w_{j1}, \dots, w_{jl})$, while the announced result consists of the coordinate-wise totals $(\sum_{j \in \mathcal{A}} w_{j1}, \dots, \sum_{j \in \mathcal{A}} w_{jl})$ over the accepted voters. This model directly captures bounded multi-option score voting and includes binary yes-or-no voting and unrestricted approval voting as special cases. It applies, for example, when DAO members privately evaluate several grant proposals, prioritize funding targets by scores, or express different levels of support for multiple policy options.

Realizing this functionality on a public blockchain introduces four closely related challenges.

Public Verifiability and Ballot Integrity: Every accepted ballot and tallying message should be publicly checkable. In particular, ballot verification must establish that every score lies in the election’s announced domain and that the range proof, masked score commitment, and encrypted sharing encode the same underlying ballot. Otherwise, a voter could submit individually well-formed but mutually inconsistent components and thereby corrupt the result.

Aggregate-only Privacy and Fairness: The public transcript should hide each voter’s score vector and prevent disclosure of a partial aggregate before the voting deadline. After tallying, it should reveal only the announced aggregate and information logically implied by that aggregate. Ballot secrecy is distinct from identity–ballot unlinkability [23], [24]: the latter additionally requires an anonymous credential and submission layer and is not provided by the cryptographic ballot alone. Fairness also requires honest talliers to withhold their shares until the deadline [13].

Decentralized and Robust Completion: No single online administrator should control whether the result can be recovered. Such an authority may otherwise appear as a key-generation authority [14], an election administrator [11], or a ballot converter [35]. At the same time, requiring voters to return during tallying makes completion fragile in an open and geographically distributed governance process. A suitable protocol should therefore tolerate the stated number of faulty or absent talliers and allow voters to leave after their ballots have been accepted [32].

Predictable Scaling with Governance Participation: A governance election may involve substantially more voters than options or talliers. Verification must necessarily be per-

Liang Zhang and Xingyu Wu are with School of Cyberspace Security (School of Cryptology), Hainan University, Haikou, 570228, China. Liang Zhang is also with Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology. (e-mail: brillasm@gmail.com; 22210839000029@hainanu.edu.cn.)

Haibin Kan is with College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200433, China. Haibin Kan is also with Shanghai Engineering Research Center of Blockchain, Shanghai 200433, China and Shanghai Institute for Mathematics and Interdisciplinary Sciences, Shanghai, 200433, China. (email: hbkan@fudan.edu.cn)

Jiheng Zhang is with Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology. (e-mail: jiheng@ust.hk)

*Corresponding author

formed for each submitted ballot, so the total voting-phase cost grows with the number m of accepted voters. Nevertheless, accepting a new ballot should not require reprocessing earlier ballots, and post-deadline finalization should operate on a compact active aggregate rather than on all m ballot transcripts. Thus, the relevant scalability goal is constant marginal work with respect to prior accepted voters and an active finalization state independent of m , rather than constant total election cost.

Existing privacy-preserving voting protocols realize different subsets of these requirements. Blockchain boardroom protocols provide public verification and, in some cases, self-tallying, but their recovery paths may require continued voter participation or retain other abort and scalability limitations [11], [12]. Score-voting systems add non-binary ballot validity, but may rely on additional trusted roles or defer m -dependent work until tallying [13], [14], [35]. Threshold and PVSS-based constructions move recovery from voters to a set of shareholders, but representative constructions primarily target binary or candidate-wise binary ballots [10], [36]. These trade-offs, summarized in Table II, motivate a construction that combines bounded score validity, public verification, online aggregation, and robust threshold completion.

To address these challenges, we introduce range-bound publicly verifiable secret sharing (RB-PVSS). An RB-PVSS ballot binds encrypted polynomial shares, masked score commitments, and one range proof per coordinate to the same private score vector. Our construction combines SCRAPE PVSS [29] with a modified constant-size range proof [39] and packs l independently bounded scores into one sharing polynomial. The smart contract verifies each ballot before atomically updating the encrypted-share and masked-score aggregates. Once voting closes, any t valid tallier shares allow every observer to reconstruct the aggregate masks and recover the announced score totals. Voters do not participate in this finalization process.

This design targets additive aggregation of independently bounded score coordinates. Voting rules that impose cross-coordinate constraints, such as fixed-total or ranking requirements, require an additional joint zero-knowledge relation, while non-additive rules require different tallying logic. These extensions are outside the present construction. The protocol also does not by itself provide identity-ballot unlinkability, which must be supplied by the application's credential and transaction-submission layer.

Our contributions are summarized as follows:

- 1) We define RB-PVSS, which binds a packed PVSS sharing, masked score commitments, and independently bounded score proofs into one publicly verifiable ballot relation. This formulation directly supports private multi-option score vectors over a public integer domain.
- 2) We construct RB-PVSS from SCRAPE PVSS and a modified constant-size range proof. We prove correctness, range-bound soundness, packed-vector privacy, and aggregation correctness under the stated threshold and setup assumptions.

- 3) We build a publicly verifiable threshold score-voting protocol for decentralized governance. The protocol provides ballot integrity, aggregate-only disclosure, fairness before the deadline, fault-tolerant completion, online aggregation, and vote-and-go. Once any t valid tallier shares are published, every observer can reproduce the result without relying on a single online tallying authority.
- 4) We implement the construction in Go and Solidity and evaluate its off-chain computation, transcript size, and on-chain gas consumption. We also compare it with representative privacy-preserving and score-voting schemes. For fixed (n, l, t) , the marginal cost of accepting a ballot is independent of the number of previously accepted voters, and finalization operates on an active aggregate whose size is independent of m . The results identify the intended deployment regime: a reusable setup, a moderate score domain, and substantially more voters than the reconstruction threshold.

II. PRELIMINARIES

Let λ be the security parameter and q a prime. We use cyclic groups $\mathcal{G}_0, \mathcal{G}_1, \mathcal{G}_T$ of order q , an efficiently computable non-degenerate pairing $e : \mathcal{G}_0 \times \mathcal{G}_1 \rightarrow \mathcal{G}_T$, and a hash function $H : \{0, 1\}^* \rightarrow \mathcal{Z}_q$. The setup algorithms below choose independent generators $g_0, h_0 \in \mathcal{G}_0$ and $g_1 \in \mathcal{G}_1$. Throughout, a closed interval used as a score or decoding domain contains only integers. Whenever an integer appears in a group exponent or an operation over $\mathcal{Z}_q$, it is identified with its residue class modulo q .

A. Blockchain and DAO

A blockchain consists of chronologically ordered blocks, each containing the hash value of its preceding block, thereby ensuring immutability and integrity. Each block encapsulates a set of transactions, forming a secure and transparent ledger. Blockchain not only underpins cryptocurrencies but also enables innovative applications such as DAOs. DAOs create self-governing entities and operate without centralized control, relying instead on smart contracts and community-driven decision-making processes. A smart contract is a program that encodes predefined rules and principles, enabling automated and trustless execution of agreements. This capability is particularly transformative for DAO governance, as it allows for transparent and tamper-proof management of organizational rules. Ethereum [26] is one of the most widely recognized platforms, with the Ethereum Virtual Machine (EVM) serving as its smart contract execution engine. Computational and storage resources required in EVM are quantified in units of gas. Gas acts as a measure of the computational effort needed to process operations. Ethereum's native digital currency, Ether, is used to pay for these gas fees and 1 Gwei=10⁻⁹ Ether.

B. Publicly Verifiable Secret Sharing

A (n, t) publicly verifiable secret sharing (PVSS) protocol [10], [29] enables dealer j to hide a secret among n shareholders indexed by N , where any subset of at least t shareholders can reconstruct the secret. The definitions of PVSS.Share

and PVSS.Recon are slightly modified to demonstrate that $l < t$ independent secrets can be shared, where $t - 1$ is the maximum polynomial degree. Let $D = \{-l+1, \dots, 0\}$ denote the packed positions and $N = \{1, \dots, n\}$ the shareholder positions.

Figure 1 introduces the PVSS [29] algorithms that enable dealer j to share a polynomial $p_j \in \mathcal{Z}_q[X]_{<t}$ among the n shareholders.

Functionality SCRAPE PVSS

$(pp_1, \{(sk_i, pk_i)\}_{i \in N}) \leftarrow \text{PVSS.Setup}(1^\lambda, n, t, l) :$
 $g_0, h_0 \xleftarrow{R} \mathcal{G}_0$
 $sk_i \xleftarrow{R} \mathcal{Z}_q^*, pk_i \leftarrow g_0^{sk_i} \text{ for } i \in N$
 $pp_1 \leftarrow (g_0, h_0, n, t, l)$
 $(\{v_{ji}\}_{i \in D \cup N}, \{c_{ji}\}_{i \in N}, \phi_j) \leftarrow \text{PVSS.Share}(p_j(\cdot), h_0, \{pk_i\}_{i \in N}) :$
 $v_{ji} = h_0^{p_j(i)} \text{ for } i \in D \cup N$
 $c_{ji} = pk_i^{p_j(i)} \text{ for } i \in N$
 $r_{ji}^\phi \xleftarrow{R} \mathcal{Z}_q, A_{ji}^\phi \leftarrow h_0^{r_{ji}^\phi}, B_{ji}^\phi \leftarrow pk_i^{r_{ji}^\phi} \text{ for } i \in N$
 $\chi_j \leftarrow H(\{v_{ji}, c_{ji}, A_{ji}^\phi, B_{ji}^\phi\}_{i \in N})$
 $z_{ji}^\phi \leftarrow r_{ji}^\phi - \chi_j p_j(i) \text{ for } i \in N$
 $\phi_j \leftarrow (\chi_j, \{A_{ji}^\phi, B_{ji}^\phi, z_{ji}^\phi\}_{i \in N})$
 $(0/1) = \text{PVSS.DVerify}(h_0, \{v_{ji}\}_{i \in D \cup N}, \{c_{ji}\}_{i \in N}, \phi_j, \{pk_i\}_{i \in N}) :$
 $A_{ji}^\phi = h_0^{z_{ji}^\phi} v_{ji}^{\chi_j}, B_{ji}^\phi = pk_i^{z_{ji}^\phi} c_{ji}^{\chi_j} \text{ for } i \in N$
 $\chi_j \stackrel{?}{=} H(\{v_{ji}, c_{ji}, A_{ji}^\phi, B_{ji}^\phi\}_{i \in N})$
 Sample a codeword $\mathbf{y} = (y_i)_{i \in D \cup N} \xleftarrow{R} \text{RS}_{D \cup N, t}^\perp$ [27]
 $\prod_{i \in D \cup N} v_{ji}^{y_i} \stackrel{?}{=} 1$
 $(sh_{ji}, \varphi_{ji}) \leftarrow \text{PVSS.Decrypt}(c_{ji}, sk_i) :$
 $sh_{ji} = c_{ji}^{1/sk_i}$
 $r_{ji}^\varphi \xleftarrow{R} \mathcal{Z}_q, A_{ji}^\varphi \leftarrow g_0^{r_{ji}^\varphi}, B_{ji}^\varphi \leftarrow sh_{ji}^{r_{ji}^\varphi}$
 $\chi_{ji} \leftarrow H(pk_i, c_{ji}, A_{ji}^\varphi, B_{ji}^\varphi)$
 $z_{ji}^\varphi \leftarrow r_{ji}^\varphi - \chi_{ji} sk_i, \varphi_{ji} \leftarrow (A_{ji}^\varphi, B_{ji}^\varphi, \chi_{ji}, z_{ji}^\varphi)$
 $(0/1) \leftarrow \text{PVSS.PVerify}(g_0, pk_i, sh_{ji}, c_{ji}, \varphi_{ji}) :$
 $A_{ji}^\varphi = g_0^{z_{ji}^\varphi} pk_i^{\chi_{ji}}, B_{ji}^\varphi = sh_{ji}^{z_{ji}^\varphi} c_{ji}^{\chi_{ji}}$
 $\chi_{ji} \stackrel{?}{=} H(pk_i, c_{ji}, A_{ji}^\varphi, B_{ji}^\varphi)$
 $\{g_0^{p_j(d)}\}_{d \in D} \leftarrow \text{PVSS.Recon}(pp_1, \{(k, sh_{jk})\}_{k \in \mathcal{Q}}, D) :$

$$\forall d \in D : \begin{cases} \mu_k(d) = \prod_{r \in \mathcal{Q}, r \neq k} \frac{d-r}{k-r} \\ \prod_{k \in \mathcal{Q}} (sh_{jk})^{\mu_k(d)} = g_0^{p_j(d)}, \quad \mathcal{Q} \subseteq N, |\mathcal{Q}| \geq t \end{cases}$$

Figure 1: The modified SCRAPE PVSS, which shares a polynomial $p_j(\cdot)$ and publishes $n + l$ PVSS evaluation commitments $\{v_{ji}\}_{i \in D \cup N}$.

A PVSS scheme has these security properties:

- **Correctness:** If the dealer and all shareholders are honest, PVSS.DVerify and PVSS.PVerify will always return 1, and l secrets can be recovered by PVSS.Recon.
- **Verifiability:** If PVSS.DVerify returns 1, then c_{ji} encrypts evaluation $p_j(i)$. If PVSS.PVerify returns 1, the decrypted-share encoding sh_{ji} equals $g_0^{p_j(i)}$.

- **IND1-Secrecy:** For any single target point outside the corrupted shareholder set, at most $t - 1$ decrypted shareholder evaluations reveal no information about the polynomial evaluation at that target point. This single-target guarantee does not by itself imply joint secrecy of several packed evaluations.

C. Zero Knowledge Range Proof

A zero-knowledge range proof (ZKRP) is a specific type of ZK proof system that allows a prover to prove that a secret value lies in some range, say $[a, b]$. Let $S = [a, b]$. ZKRP is an instrumental building block in voting scenarios to guarantee voters cast a ballot honestly. The ZKRP algorithms are defined as follows: The Camenisch et al. ZKRP scheme [39] proves that w lies within a specified range $[a, b]$, based on the Boneh-Boyen-style signature [40].

Functionality Constant-size ZKRP

$pp_2 \leftarrow \text{ZKRP.Setup}(1^\lambda, g_0, h_0, S) :$
 $g_1 \xleftarrow{R} \mathcal{G}_1, sk_I \xleftarrow{R} \mathcal{Z}_q \setminus \{-k : k \in S\}, pk_I \leftarrow g_1^{sk_I}$
 $pp_2 \leftarrow \{g_0, h_0, g_1, pk_I, \{\sigma_k = g_0^{1/(sk_I+k)}\}_{k \in S}\}$
 $\pi \leftarrow \text{ZKRP.Prove}(s, w, U = h_0^w g_0^s) :$
 $\beta, w', \beta', s' \xleftarrow{R} \mathcal{Z}_q$
 $E \leftarrow \sigma_w^\beta$
 $F \leftarrow e(E, g_1)^{-w'} e(g_0, g_1)^{\beta'}$
 $U' \leftarrow h_0^{w'} g_0^{s'}$
 $\chi = H(E, U, F, U')$
 $z_1 = w' - \chi w, z_\beta = \beta' - \chi \beta, z_3 = s' - \chi s$
 $\pi = \{E, F, U', \chi, z_1, z_\beta, z_3\}$
 $(0/1) \leftarrow \text{ZKRP.Verify}(\pi, U) :$
 $\chi \stackrel{?}{=} H(E, U, F, U')$
 $U' \stackrel{?}{=} U^\chi g_0^{z_3} h_0^{z_1}$
 $F \stackrel{?}{=} e(E, pk_I)^\chi \cdot e(E, g_1)^{-z_1} \cdot e(g_0, g_1)^{z_\beta}$

Figure 2: The Camenisch et al. constant-size ZKRP [39]

- **Completeness:** For any $\pi \leftarrow \text{ZKRP.Prove}(s, w, U)$, $\Pr[\text{ZKRP.Verify}(\pi, U) = 1] = 1$.
- **Soundness:** No PPT adversary can make an honest verifier accept a proof for $w \notin [a, b]$.
- **Zero-knowledge:** It is possible to simulate the interaction between an honest prover and a verifier in polynomial time, without knowing the witnesses w and s .

D. Symbols

The main symbols used in this paper are summarized in Table I.

III. SYSTEM MODEL AND SECURITY GOALS

A. Parties and Ballot Model

An election involves an initiator, a voter DAO indexed by an eligible voter set $\mathcal{V}$, a tallier DAO with n talliers indexed

Table I: Symbols in this paper

Symbol	Description
Public Setting and Election Model	
λ, q	security parameter and prime group order
$\mathcal{G}_0, \mathcal{G}_1, \mathcal{G}_T, e$	source groups, target group, and pairing
$\mathbb{Z}_q$	the finite field of prime order q
n, l	number of talliers and ballot governance options
t	RB-PVSS reconstruction threshold
D	packed evaluation positions $\{-l+1, \dots, 0\}$
N	tallier/shareholder indices $\{1, \dots, n\}$
S	per-coordinate score domain $[a, b]$; a, b are its endpoints
sk_I, pk_I	one-time range-parameter issuer key pair
$\mathcal{V}$	eligible voter-index set
m	accepted-ballot count $ \mathcal{A} $
f	maximum number of corrupted or aborting talliers
Per-Ballot RB-PVSS Objects	
$w_{jd}, \mathbf{w}_j$	score at coordinate d and multi-option score vector in S^l
$p_j(\cdot)$	voter j 's sharing polynomial of degree less than t
v_{ji}	PVSS evaluation commitment $h_0^{p_j(i)}$
c_{ji}	encrypted PVSS share for tallier i
U_{jd}	masked score commitment $g_0^{p_j(d)} h_0^{w_{jd}}$
π_{jd}	the d -th range-binding proof from voter j
ϕ_j, φ_{ji}	PVSS dealer proof and decryption-share proof
$\mathcal{I}_j$	the RB-PVSS dealer transcript of voter j
sh_{ji}	decrypted-share encoding $g_0^{p_j(i)}$
Aggregation, State, and Tallying	
$\mathcal{A}(\subseteq \mathcal{V}), \mathcal{Q}$	accepted voter indices and tallier-share indices
$\mathcal{H}_T$	honest tallier set, $\mathcal{H}_T \subseteq N$ and $ \mathcal{H}_T \geq n-f$
$p_{\mathcal{A}}(X)$	aggregate polynomial $\sum_{j \in \mathcal{A}} p_j(X)$
C_i^*	aggregate encrypted share for tallier i
U_d^*	aggregate masked score commitment $\prod_{j \in \mathcal{A}} U_{jd}$
sh_i	aggregate decrypted-share encoding $g_0^{p_{\mathcal{A}}(i)}$
$S_{\mathcal{A}}$	aggregate decoding domain $[\mathcal{A} a, \mathcal{A} b]$
$\mathbf{W}$	published aggregate vector

by N , and a smart contract. The initiator creates the proposal and fixes its public policy. Voter j , for $j \in \mathcal{V}$, submits a private score vector $\mathbf{w}_j = (w_{jd})_{d \in D} \in S^l$, where coordinates represent the ordered governance options. Tallier i , holding (sk_i, pk_i) , acts as an RB-PVSS shareholder. The contract is the public bulletin board and state machine: it records, verifies, and aggregates protocol messages. The ballot-dependent plaintext output is only the coordinate-wise sum of the accepted vectors. Let $\mathcal{A} \subseteq \mathcal{V}$ denote the accepted voter-index set and reserve $m := |\mathcal{A}|$ for its size, as used in the complexity analysis and experiments; no scalar is needed for $|\mathcal{V}|$.

B. Trust and Adversary Model

We consider a static probabilistic polynomial-time adversary. The initiator is honest-but-curious, voters may submit arbitrary transcripts, and at most f talliers may collude or abort. Let $\mathcal{H}_T \subseteq N$ be the honest tallier set with $|\mathcal{H}_T| \geq n-f$. Sharing polynomials have degree less than t , and the admissible parameters satisfy

$$l < t \leq n, \quad t - f \geq l, \quad n - f \geq t,$$

or equivalently $f \leq \min\{t-l, n-t\}$. The first inequality leaves enough residual polynomial degrees of freedom to hide all l packed evaluations jointly, while the second leaves enough

honest talliers to reconstruct. A convenient balanced choice is $t = \lfloor (n+l)/2 \rfloor$, for which the supported bound is $f \leq \lfloor (n-l)/2 \rfloor$. Honest talliers release decryption-share encodings only after the public deadline τ .

The blockchain provides an immutable public transcript and a common clock, and the contract correctly executes the specified state transitions. The range-issuer secret is erased after the public parameters are checked; ballot secrecy does not rely on keeping it hidden. The public generators g_0, h_0 are generated so that no party retains their discrete-log relation. The application layer provides a trusted one-to-one mapping $\text{id}_j \leftrightarrow j$: after authenticating id_j , the contract uniquely derives the corresponding voter index j . Voter authentication, eligibility, one-ballot enforcement, Sybil resistance, and transaction-layer anonymity are supplied by this application layer and are not consequences of RB-PVSS. Within this model, the adversary may attempt to submit malformed or out-of-range ballots, infer an individual ballot before or after tallying, learn an early aggregate, publish invalid decryption-share encodings, or obstruct finalization.

C. Security and Design Goals

Cryptographic goals. Integrity and public verifiability require the published result to equal the sum of exactly the accepted ballots and make every acceptance and finalization step publicly checkable. Ballot secrecy hides each honest vector against at most f corrupted talliers, both before tallying and afterward except for the announced aggregate and its logical implications. Fairness excludes a pre-deadline partial tally. Robust threshold tallying rejects invalid messages, completes within the stated fault bound, and lets any observer reproduce the result from t valid decryption-share encodings.

Design goals and boundaries. The protocol supports multi-option non-binary scores and vote-and-go: after acceptance, a voter sends no further message. Per-voter work is independent of the accepted-ballot count m , although total verification and aggregation remain linear in m . Tallying needs no online trusted authority. The cryptographic transcript itself contains no voter identifier, but protocol-level anonymity requires an unlinkable submission or credential layer.

IV. RANGE-BOUND PVSS

A. Syntax

We introduce *range-bound publicly verifiable secret sharing* (RB-PVSS), a PVSS primitive whose shared values are additionally certified to lie in a public interval. Let $S = [a, b]$, let n be the number of shareholders, let t be the reconstruction threshold, and let $l < t$ be the number of packed values. The sharing polynomial has degree strictly smaller than t . We use the public positions $D = \{-l+1, \dots, 0\}$ for the packed values and $N = \{1, \dots, n\}$ for the shareholders.

Definition 1 (Range-bound PVSS): An RB-PVSS scheme consists of the following algorithms.

- $(pp, \{sk_i\}_{i \in N}) \leftarrow \text{RB-PVSS.Setup}(1^\lambda, n, t, l, S)$
generates the PVSS and range-proof parameters

and the shareholders' keys. In particular, $pp = (pp_1, pp_2, \{pk_i\}_{i \in N}, N, D, t, S)$.

- $\mathcal{I}_j \leftarrow \text{RB-PVSS.Share}(pp, w_j)$, for $w_j = (w_{jd})_{d \in D} \in S^l$, outputs $\mathcal{I}_j = (\{v_{ji}\}_{i \in D \cup N}, \{c_{ji}\}_{i \in N}, \phi_j, \{U_{jd}, \pi_{jd}\}_{d \in D})$. Here U_{jd} is the masked score commitment to w_{jd} , masked by the sharing-polynomial evaluation at the packed position d . The proof π_{jd} certifies both $w_{jd} \in S$ and the consistency of this mask with the PVSS transcript; the precise group relation is given in Eq. (1).
- $\text{RB-PVSS.DVerify}(pp, \mathcal{I}_j)$ publicly verifies the complete dealer transcript and returns a bit.
- $(sh_{ji}, \varphi_{ji}) \leftarrow \text{RB-PVSS.Decrypt}(pp, c_{ji}, sk_i)$ decrypts a ciphertext and produces its decryption-share proof.
- $\text{RB-PVSS.PVerify}(pp, i, sh_{ji}, c_{ji}, \varphi_{ji})$ performs decryption-share verification and returns a bit.
- $w \leftarrow \text{RB-PVSS.Recon}(pp, \{(i, sh_i)\}_{i \in Q}, \{U_d\}_{d \in D}, S')$ reconstructs the packed values from a set of decrypted-share encodings, where $Q \subseteq N$, $|Q| \geq t$, and $S' \subseteq \mathbb{Z}$ is a public decoding interval. It reconstructs the masks, removes them from $\{U_d\}_{d \in D}$, and returns the uniquely decoded vector; it returns $\perp$ if any bounded discrete logarithm has no solution in S' . For a single dealer instance, $sh_i = sh_{ji}$, $U_d = U_{jd}$, and $S' = S$.

The validity relation is the defining feature of the primitive. For a dealer transcript, define the public statement and witness once as

$$x_j = (pp, \{v_{ji}\}_{i \in D \cup N}, \{c_{ji}\}_{i \in N}, \{U_{jd}\}_{d \in D}), \\ \omega_j = (p_j, \{w_{jd}\}_{d \in D}).$$

Its validity is captured by the single relation

$$\mathcal{R}_{\text{RB}}(x_j, \omega_j) = 1 \iff \\ p_j \in \mathcal{Z}_q[X]_{<t}, \quad w_{jd} \in S \quad (d \in D), \\ v_{ji} = h_0^{p_j(i)} \quad (i \in D \cup N), \\ c_{ji} = pk_i^{p_j(i)} \quad (i \in N), \\ U_{jd} = h_0^{w_{jd}} g_0^{p_j(d)} \quad (d \in D). \quad (1)$$

Thus, acceptance certifies not only a valid PVSS polynomial and a valid range proof, but also that the mask in every U_{jd} is the evaluation of that same polynomial.

B. Construction

Figure 3 directly realizes a joint proof of knowledge for Eq. (1):

$$\text{ZKPoK}\{\omega_j : \mathcal{R}_{\text{RB}}(x_j, \omega_j) = 1\}.$$

The PVSS component certifies one common sharing polynomial, while each range-binding proof certifies that a packed value lies in S and uses the corresponding evaluation of that same polynomial as its mask. The verifier therefore treats the PVSS transcript and all coordinate proofs as one atomic object.

The main intuition is to reuse the packed evaluations $\{p_j(d)\}_{d \in D}$ as masks for all l values. This preserves the threshold structure of PVSS while avoiding a second sharing polynomial. After valid instances are aggregated, any t talliers

Functionality Construction of Range-Bound PVSS

$$\begin{aligned} & (pp, \{sk_i\}_{i \in N}) \leftarrow \text{RB-PVSS.Setup}(1^\lambda, n, t, l, S) : \\ & \quad (pp_1, \{(sk_i, pk_i)\}_{i \in N}) \leftarrow \text{PVSS.Setup}(1^\lambda, n, t, l) \\ & \quad pp_2 \leftarrow \text{ZKRP.Setup}(1^\lambda, g_0, h_0, S) \\ & \quad pp \leftarrow (pp_1, pp_2, \{pk_i\}_{i \in N}, N, D, t, S) \\ & \mathcal{I}_j \leftarrow \text{RB-PVSS.Share}(pp, w_j) : \\ & \quad \text{Sample } p_j \leftarrow \mathcal{Z}_q[X]_{<t} \\ & \quad (\{v_{ji}\}_{i \in D \cup N}, \{c_{ji}\}_{i \in N}, \phi_j) \leftarrow \text{PVSS.Share}(p_j, h_0, \{pk_i\}_{i \in N}). \\ & \quad \text{For each } d \in D, \text{ set } U_{jd} = h_0^{w_{jd}} g_0^{p_j(d)} : \\ & \quad \quad \beta_{jd}, w'_{jd}, \beta'_{jd}, s'_{jd} \xleftarrow{R} \mathcal{Z}_q \\ & \quad \quad E_{jd} \leftarrow \sigma_{w'_{jd}}^{\beta_{jd}} \\ & \quad \quad F_{jd} \leftarrow e(E_{jd}, g_1)^{-w'_{jd}} e(g_0, g_1)^{\beta'_{jd}} \\ & \quad \quad U'_{jd} \leftarrow h_0^{w'_{jd}} g_0^{s'_{jd}} \\ & \quad \quad C'_{jd} \leftarrow h_0^{s'_{jd}} \\ & \quad \quad \chi_{jd} \leftarrow H(E_{jd}, U_{jd}, v_{jd}, F_{jd}, U'_{jd}, C'_{jd}) \\ & \quad \quad z_{jd,1} \leftarrow w'_{jd} - \chi_{jd} w_{jd}, \quad z_{jd,\beta} \leftarrow \beta'_{jd} - \chi_{jd} \beta_{jd} \\ & \quad \quad z_{jd,3} \leftarrow s'_{jd} - \chi_{jd} p_j(d) \\ & \quad \quad \pi_{jd} \leftarrow (E_{jd}, F_{jd}, U'_{jd}, C'_{jd}, \chi_{jd}, z_{jd,1}, z_{jd,\beta}, z_{jd,3}) \\ & (0/1) \leftarrow \text{RB-PVSS.DVerify}(pp, \mathcal{I}_j) : \\ & \quad \text{PVSS.DVerify}(h_0, \{v_{ji}\}_{i \in D \cup N}, \{c_{ji}\}_{i \in N}, \phi_j, \{pk_i\}_{i \in N}) \stackrel{?}{=} 1 \\ & \quad \text{for every } d \in D: \\ & \quad \quad \chi_{jd} \stackrel{?}{=} H(E_{jd}, U_{jd}, v_{jd}, F_{jd}, U'_{jd}, C'_{jd}) \\ & \quad \quad C'_{jd} \stackrel{?}{=} v_{jd}^{\chi_{jd}} h_0^{z_{jd,3}} \\ & \quad \quad U'_{jd} \stackrel{?}{=} U_{jd}^{\chi_{jd}} g_0^{z_{jd,3}} h_0^{z_{jd,1}} \\ & \quad \quad F_{jd} \stackrel{?}{=} e(E_{jd}, pk_i)^{\chi_{jd}} e(E_{jd}, g_1)^{-z_{jd,1}} e(g_0, g_1)^{z_{jd,\beta}} \\ & (sh_{ji}, \varphi_{ji}) \leftarrow \text{RB-PVSS.Decrypt}(pp, c_{ji}, sk_i) : \\ & \quad (sh_{ji}, \varphi_{ji}) \leftarrow \text{PVSS.Decrypt}(c_{ji}, sk_i) \\ & (0/1) \leftarrow \text{RB-PVSS.PVerify}(pp, i, sh_{ji}, c_{ji}, \varphi_{ji}) : \\ & \quad \text{PVSS.PVerify}(g_0, pk_i, sh_{ji}, c_{ji}, \varphi_{ji}) \stackrel{?}{=} 1 \\ & \{w_{jd}\}_{d \in D} \leftarrow \text{RB-PVSS.Recon}(pp, \{(i, sh_{ji})\}_{i \in Q}, \{U_{jd}\}_{d \in D}, S) : \\ & \quad \{M_{jd}\}_{d \in D} \leftarrow \text{PVSS.Recon}(pp_1, \{(i, sh_{ji})\}_{i \in Q}, D) \\ & \quad \text{For every } d \in D: \\ & \quad \quad Y_{jd} \leftarrow U_{jd}/M_{jd} \\ & \quad \quad w_{jd} \leftarrow \text{DLog}_{h_0}(Y_{jd}; S) \end{aligned}$$

Figure 3: Direct construction of RB-PVSS. All RB-PVSS interfaces are shown explicitly; sharing outputs the PVSS transcript and $\{U_{jd}, \pi_{jd}\}_{d \in D}$ together, while reconstruction decodes the shared values.

provide enough decrypted-share encodings for reconstruction to remove the aggregate masks and decode the tally in its public range.

Only sharing and dealer verification need the additional range-binding logic. Decryption and decryption-share verification retain the behavior of their underlying PVSS counterparts. Reconstruction first invokes the underlying PVSS reconstruction to obtain the masks, then removes them from the supplied commitments and performs bounded discrete-log decoding.

C. Security of the Construction

We require correctness, range-bound soundness, and privacy. Correctness means that an honestly generated instance is accepted and that any t valid decrypted-share encodings recover its l values. Range-bound soundness means that every accepting dealer transcript has witnesses satisfying all three equalities in Eq. (1); in particular, the range proof and the encrypted shares cannot refer to different masks. Privacy is defined by indistinguishability under the system corruption bound: after choosing two vectors in S^l , an adversary corrupting at most f shareholders cannot distinguish their dealer transcripts, provided $t - f \geq l$, except for information explicitly released by a later aggregate tally.

Theorem 1 (Correctness and public verifiability): For every $w_j \in S^l$, an instance output by RB-PVSS.Share is accepted by RB-PVSS.DVerify. Moreover, any t valid decrypted-share encodings, together with $\{U_{jd}\}_{d \in D}$ and decoding interval S , reconstruct w_j .

Proof. PVSS correctness gives $v_{ji} = h_0^{p_j(i)}$, $c_{ji} = pk_i^{p_j(i)}$, and successful reconstruction of $g_0^{p_j(d)}$. For each d , substituting $z_{jd,3} = s'_{jd} - \chi_{jd} p_j(d)$ gives

$$v_{jd}^{\chi_{jd}} h_0^{z_{jd,3}} = h_0^{\chi_{jd} p_j(d) + s'_{jd} - \chi_{jd} p_j(d)} = C'_{jd}.$$

The ordinary range-proof equations (Section II-C) hold by completeness. Therefore RB-PVSS.DVerify accepts, and the reconstructed mask is precisely $M_{jd} = g_0^{p_j(d)}$. Hence $U_{jd}/M_{jd} = h_0^{w_{jd}}$, and bounded discrete-log decoding over S returns the unique value w_{jd} for every $d \in D$. $\square$

Theorem 2 (Range-bound soundness): Except with negligible probability, if RB-PVSS.DVerify accepts $\mathcal{I}_j$, then there exist a single polynomial $p_j \in \mathcal{Z}_q[X]_{<t}$ and values $w_{jd} \in S$ satisfying Eq. (1) simultaneously.

Proof sketch. PVSS verifiability implies that the accepted commitments and ciphertexts encode evaluations of one degree- $(t-1)$ polynomial p_j . By the knowledge soundness of the modified range proof, an accepting transcript for coordinate d yields $w_{jd} \in S$ and an opening exponent s_{jd} of U_{jd} . Its additional Sigma equation uses the same response for U_{jd} and v_{jd} ; special soundness therefore gives $s_{jd} = p_j(d)$. Applying this argument to every $d \in D$ establishes the joint relation. Otherwise, an adversary would break either PVSS verifiability or the soundness of the range proof. $\square$

Theorem 3 (Packed-vector privacy): Assume the IND1 secrecy of the underlying PVSS, the zero-knowledge property of the range proof, and the hardness of relating discrete logarithms across the independent bases g_0 and h_0 . Against an adversary corrupting at most f shareholders, where $t - f \geq l$, RB-PVSS hides an honest dealer's vector before tallying. After tallying it reveals no more than the announced aggregate vector, public metadata, and their logical implications.

Proof sketch. Evaluations of a uniformly random degree- $(t-1)$ polynomial at any set of at most t distinct points are jointly uniform. The corrupted shareholder positions are disjoint from D , and together these at most $f + l \leq t$ positions are distinct. Consequently, conditioning on any at

most f corrupted shareholder evaluations leaves the packed mask vector $\{p_j(d)\}_{d \in D}$ jointly uniform in $\mathcal{Z}_q^l$. This is the multi-target entropy argument that is absent from ordinary single-target IND1 secrecy. The PVSS secrecy property hides undecrypted shareholder evaluations and the encrypted shares. A standard cross-base DDH hybrid then replaces the jointly uniform masks in the U_{jd} 's, after which the two challenge vectors induce indistinguishable commitments; zero-knowledge simulates the π_{jd} 's without their openings.

At tally time, only shares of the aggregate polynomial are released. For the honest dealers, their mask vectors remain jointly uniform subject to their disclosed sum, so the same hybrid hides each individual vector modulo the announced aggregate and auxiliary information. If that information already determines an individual ballot, the resulting inference is a logical implication of the permitted output rather than additional protocol leakage. $\square$

D. Homomorphic Aggregation of RB-PVSS Instances

Let $\{\mathcal{I}_j\}_{j \in \mathcal{A}}$ be a set of accepted, context- and parameter-compatible RB-PVSS instances. Besides aggregating the encrypted shares, the algorithm aggregates the masked score commitments:

$$\begin{aligned} (\{C_i^*\}_{i \in N}, \{U_d^*\}_{d \in D}) &\leftarrow \text{Aggregate}(\{\mathcal{I}_j\}_{j \in \mathcal{A}}): \\ C_i^* &\leftarrow \prod_{j \in \mathcal{A}} c_{ji}, \quad (i \in N), \\ U_d^* &\leftarrow \prod_{j \in \mathcal{A}} U_{jd}, \quad (d \in D). \end{aligned} \quad (2)$$

Lemma 1 (Aggregation correctness): For accepted instances, $\{C_i^*\}_{i \in N}$ encrypt evaluations of the aggregate polynomial $p_{\mathcal{A}}(X) = \sum_{j \in \mathcal{A}} p_j(X)$. Then $U_d^* = h_0^{\sum_{j \in \mathcal{A}} w_{jd}} g_0^{p_{\mathcal{A}}(d)}$. Consequently, the aggregate value belongs to $[\mathcal{A}|a, \mathcal{A}|b]$.

Proof. The claim follows by multiplying the equalities in Eq. (1). Polynomial addition does not increase the degree, and the sum of $|\mathcal{A}|$ values from $[a, b]$ lies in $[\mathcal{A}|a, \mathcal{A}|b]$. $\square$

Figure 4 depicts this homomorphic aggregation, applied in later score voting scenarios. Tallyer i decrypts $(\text{sh}_i, \varphi_i) \leftarrow \text{RB-PVSS.Decrypt}(pp, C_i^*, sk_i)$, where $\text{sh}_i = g_0^{p_{\mathcal{A}}(i)}$. Let $S_{\mathcal{A}} = [\mathcal{A}|a, \mathcal{A}|b]$. We require $|\mathcal{A}|(b-a) < q$, so reduction modulo q is injective on this decoding interval. From any t valid decrypted-share encodings, $\text{RB-PVSS.Recon}(pp, \{(i, \text{sh}_i)\}_{i \in \mathcal{Q}}, \{U_d^*\}_{d \in D}, S_{\mathcal{A}})$ returns $\{\sum_{j \in \mathcal{A}} w_{jd}\}_{d \in D}$.

V. THE SYSTEM IN DETAIL

A. Protocol Overview and State

The system instantiates the entities and adversarial model of Section III-B with RB-PVSS. Its native ballot is an independent-range score vector $w_j \in S^l$, and its only ballot-dependent plaintext output is the coordinate-wise sum $W = \sum_{j \in \mathcal{A}} w_j$ of the accepted ballots. Each voter acts as an RB-PVSS dealer, whereas talliers are its shareholders.

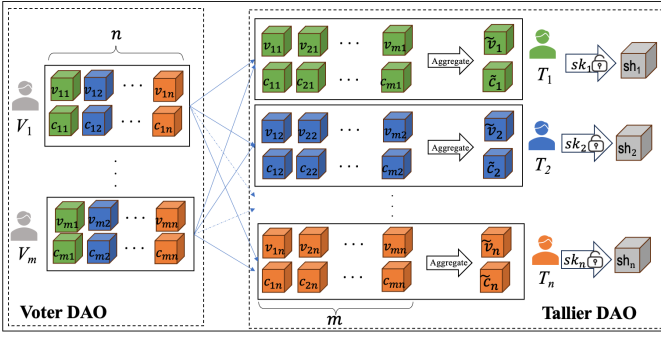


Figure 4: Aggregation of accepted RB-PVSS instances for n talliers, where voter DAO and tallier DAO are separated. The talliers can decrypt the aggregated ciphertexts and reconstruct the sum of the accepted ballots.

For an election iid , the contract state is

$$st = (\text{phase}, \mathcal{A}, \mathcal{Q}, \{C_i^*\}_{i \in N}, \{U_d^*\}_{d \in D}),$$

where phase progresses from Open to Closed, and then to Finalized. Here $\mathcal{A}$ contains accepted voter indices, and $\mathcal{Q} \subseteq N$ contains accepted tallier-share indices. Initially $\mathcal{A} = \mathcal{Q} = \emptyset$, $C_i^* = 1_{\mathcal{G}_0}$, and $U_d^* = 1_{\mathcal{G}_0}$. Every accepted ballot preserves the public aggregation invariants

$$C_i^* = \prod_{j \in \mathcal{A}} c_{ji}, \quad U_d^* = \prod_{j \in \mathcal{A}} U_{jd}.$$

We write $\text{Aggregate}(st, \mathcal{I}_j)$ for this incremental form of Eq. (2).

Figure 5 shows the protocol sequence and the party that invokes each RB-PVSS algorithm.

B. SETUP: Parameter Generation and Election Creation

The reusable setup $(pp, \{sk_i\}_{i \in N}) \leftarrow \text{RB-PVSS.Setup}(1^\lambda, n, t, l, S)$ fixes a compatible tallier set, threshold, number of coordinates, and score domain. The joint output notation abstracts a distributed setup: pp , including $\{pk_i\}_{i \in N}$, is public, while each tallier locally generates and retains only its own sk_i . The range-parameter issuer generates the public range-signature table and erases sk_I after the table is checked. Talliers publish their locally generated pk_i . Compatible elections may reuse these parameters.

For each election, create binds iid to pp and a policy containing the ordered options indexed by D , score domain, deadline τ , and eligibility rule. The contract checks parameter compatibility, initializes st , and opens voting.

C. VOTING: Verification and Online Aggregation

An application-layer predicate $\text{Eligible}(iid, id_j)$ determines who may vote. Using the trusted mapping $id_j \leftrightarrow j$, the contract derives j and rejects the ballot whenever $j \in \mathcal{A}$. This interface covers both permissioned credentials and open-membership policies, but Sybil resistance and identity anonymity are outside RB-PVSS.

An eligible voter computes $\mathcal{I}_j \leftarrow \text{RB-PVSS.Share}(pp, w_j)$ and submits $\text{vote}(iid, id_j, \mathcal{I}_j)$. If $j \notin \mathcal{A}$ and $\text{RB-PVSS.DVerify}(pp, \mathcal{I}_j) = 1$, the contract atomically adds j to $\mathcal{A}$ and invokes $\text{Aggregate}(st, \mathcal{I}_j)$; otherwise, neither $\mathcal{A}$ nor the aggregates change. Thus only one well-formed, range-bound ballot per authenticated voter affects the tally.

D. TALLYING: Closure and Finalization

At $\text{block.ts} > \tau$, or at the deterministic closing condition bound to the proposal, $\text{close}(iid)$ changes the election to Closed and rejects later ballots. Tallier i then computes

$$(\text{sh}_i, \varphi_i) \leftarrow \text{RB-PVSS.Decrypt}(pp, C_i^*, sk_i)$$

and submits the pair. The contract adds i to $\mathcal{Q}$ at most once and only if $\text{RB-PVSS.PVerify}(pp, i, \text{sh}_i, C_i^*, \varphi_i) = 1$.

Let $S_{\mathcal{A}} = [\mathcal{A}|a, \mathcal{A}|b]$. The contract requires $|\mathcal{A}|(b - a) < q$, which prevents modular wraparound and makes bounded decoding unique. Once $|\mathcal{Q}| \geq t$, the public finalization call is

$$\mathbf{W} \leftarrow \text{RB-PVSS.Recon}(pp, \{(i, \text{sh}_i)\}_{i \in \mathcal{Q}}, \{U_d^*\}_{d \in D}, S_{\mathcal{A}}).$$

This call reconstructs the masks and returns the aggregate. The contract executes RB-PVSS.PVerify and the group-operation portion of RB-PVSS.Recon ; any observer can complete the public decoding and verify each result from the reconstructed group element.

E. Native Scope and Extensions

The construction natively proves only the coordinate-wise relation $w_j \in S^l$. It therefore directly supports non-binary multi-option scores, yes-or-no voting with $l = 1, S = \{0, 1\}$, and unrestricted approval vectors with $S = \{0, 1\}$, all of which are common in DAO platforms such as Snapshot [15].

Voting rules that relate different coordinates require an additional zero-knowledge relation. Examples include $\sum_{d \in D} w_{jd} = 1$ for single-choice voting, a sum constraint for a fixed approval budget, a quadratic constraint under a voter-specific public budget, and distinctness or permutation constraints for ranked ballots. Checking these private relations directly in the contract would reveal the ballot; supporting them therefore requires extending the proof relation and its security analysis. Instant-runoff tallying additionally needs non-additive tally logic and is outside the present construction.

VI. SECURITY ANALYSIS

Under the model of Section III-B, we first prove tally correctness and then lift the RB-PVSS guarantees to the system goals.

Lemma 2 (Tally correctness): Finalization returns $\mathbf{W} = (\sum_{j \in \mathcal{A}} w_{jd})_{d \in D}$.

Proof. By Lemma 1, C_i^* encrypts an evaluation of $p_{\mathcal{A}}(X) = \sum_{j \in \mathcal{A}} p_j(X)$, while $U_d^* = h_0^{\sum_{j \in \mathcal{A}} w_{jd}} g_0^{p_{\mathcal{A}}(d)}$. Any t valid shares reconstruct $g_0^{p_{\mathcal{A}}(d)}$; removing it and decoding over $S_{\mathcal{A}}$ gives the claimed coordinate sum. $\square$

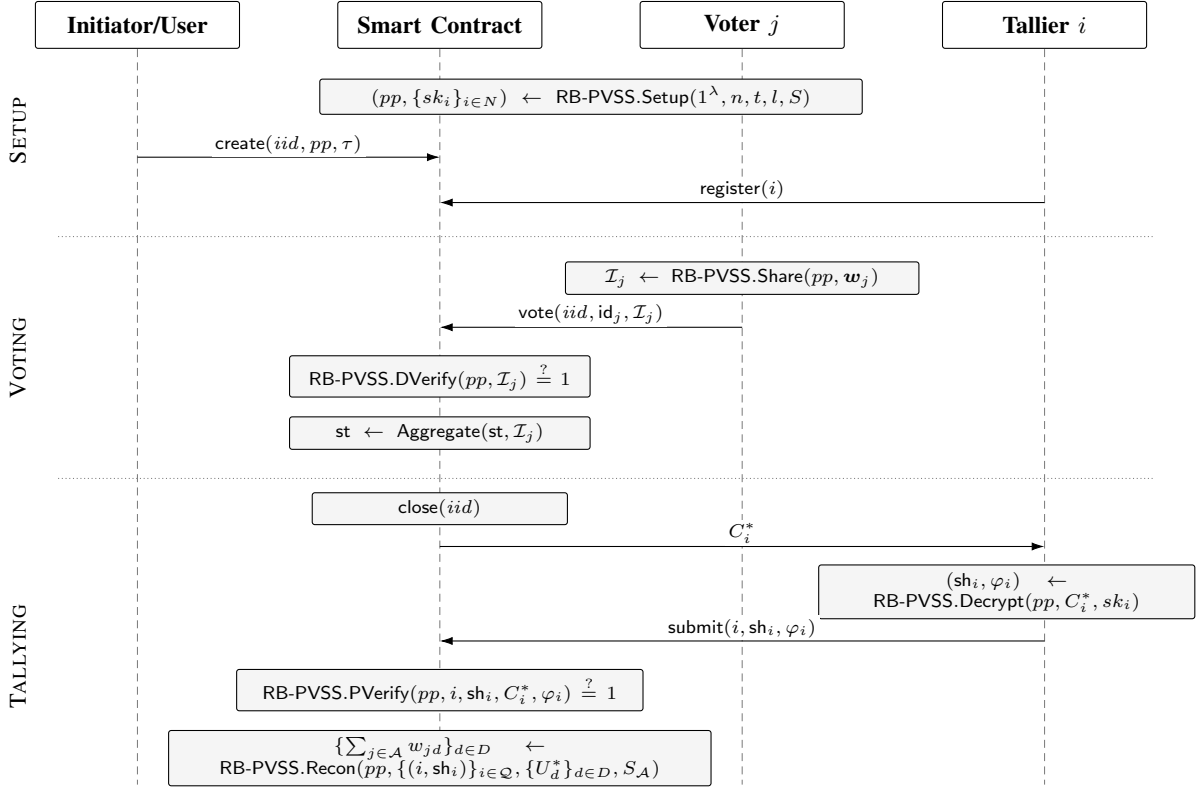


Figure 5: Protocol sequence using the RB-PVSS interfaces.

A. Reduction of System Threats

The adversarial behaviors in Section III-B reduce to three primitive guarantees. Range-bound soundness (Theorem 2) excludes malformed, inconsistent, and out-of-range ballots because aggregation follows only after RB-PVSS.DVerify accepts. Packed-vector privacy (Theorem 3) covers the initiator's setup view, individual-ballot inference, and post-tally leakage. The same guarantee and the threshold exclude a premature aggregate: before the deadline the adversary has at most f shares, and $t - f \geq l$ leaves all l aggregate masks jointly hidden. Finally, corrupted talliers cannot prevent finalization because $|\mathcal{H}_T| \geq n - f \geq t$, so the honest talliers alone can supply t valid shares. The following theorems state the resulting system properties without repeating these reductions.

B. Property Analysis

Theorem 4 (Integrity and public verifiability): Except with negligible probability, the published tally is the coordinate-wise sum of exactly the distinct ballots accepted by the contract, and any observer can verify every accepted ballot, every accepted decryption-share encoding, and the reconstructed result.

Proof. For every accepted instance, Theorem 2 supplies a single degree- $(t-1)$ polynomial and an in-range vector bound to it. The trusted mapping $id_j \leftrightarrow j$, the check $j \notin \mathcal{A}$, and the atomic update ensure that every authenticated voter index

occurs at most once and that no rejected record enters the aggregate. Public RB-PVSS.DVerify and RB-PVSS.PVerify checks validate, respectively, the ballot and each decryption-share encoding. Lemmas 1 and 2 then show that reconstruction from any t valid shares returns exactly the sum of the accepted vectors. All inputs to these deterministic checks are on the blockchain, so any observer can replay them and the final reconstruction. $\square$

Theorem 5 (Ballot secrecy and aggregate-only disclosure):

Against an adversary corrupting at most f talliers, with $t - f \geq l$, an honest voter's vector is hidden before the tally. Publishing the aggregate decryption shares reveals no additional information about an individual honest vector beyond the announced aggregate and what is logically implied by that aggregate and the adversary's auxiliary information.

Proof. Before tallying, this is Theorem 3. At tallying, talliers decrypt only the aggregated ciphertexts $\{C_i^*\}_{i \in N}$, and each proof φ_i is bound to its aggregate ciphertext; no decryption share for an individual ballot is published. The post-tally part of Theorem 3 therefore limits the new leakage to the vector reconstructed in Lemma 2. As usual, aggregate-only disclosure cannot hide a ballot that is already uniquely determined by the output and the adversary's side information. $\square$

Theorem 6 (Fairness): Before the voting deadline, the adversary cannot learn the partial tally of the accepted honest ballots, except with negligible probability.

Proof. Honest talliers do not decrypt before the deadline, so the adversary holds at most f shares. Since $t - f \geq l$, Theorem 3 implies that it cannot recover the aggregate mask vector and hence cannot remove those masks from $\{U_d^*\}_{d \in D}$. Learning the partial tally would therefore contradict packed-vector privacy. $\square$

Theorem 7 (Robust publicly verifiable threshold tallying):

Malformed ballots and decryption-share encodings do not change the result. Moreover, under $t - f \geq l$ and $n - f \geq t$, the honest talliers can complete the tally; once any t valid shares are published, every observer can reproduce it.

Proof. Invalid ballots and shares fail RB-PVSS.DVerify and RB-PVSS.PVerify, respectively. The honest set satisfies $|\mathcal{H}_T| \geq n - f \geq t$, so it can supply t valid shares without help from corrupted talliers. RB-PVSS correctness (Theorem 1) and aggregation correctness (Lemma 1) imply successful reconstruction. Because the accepted transcript and the t verified shares are public, any observer obtains the same result by invoking RB-PVSS.Recon. $\square$

Theorem 8 (Vote-and-go): After its ballot has been accepted, an honest voter need not send another protocol message.

Proof. The voter invokes RB-PVSS.Share and submits $\mathcal{I}_j$ only in the VOTING phase. Online aggregation is performed by the contract, and all post-deadline decryption messages are supplied by talliers. $\square$

These results justify the entries for our scheme in Table II: public verification follows from Theorem 4, aggregate-only disclosure from Theorem 5, fault-tolerant completion from Theorem 7, and the absence of post-ballot voter messages from Theorem 8.

a) Design properties and limitations.: The remaining requirements are not additional cryptographic security claims. Voter-side computation and communication depend on n and l , not on the accepted-ballot count m ; the total verification and aggregation work nevertheless grows linearly with m , as quantified in Section VIII-B. Tallying is distributed among the tallier DAO and needs no online trusted tallying authority, but the reusable range parameters must be generated and checked as specified in the SETUP phase. Finally, an RB-PVSS.Share transcript contains no voter identifier, whereas a transaction sender or an eligibility record may still link a voter to that transcript. Thus the protocol provides transcript-level unlinkability only when it is combined with an unlinkable submission or credential layer; it does not claim protocol-level voter anonymity by itself.

VII. RELATED WORK

Table II separates properties that are often conflated. Public verifiability means that an observer can check accepted ballots and the announced tally, but does not by itself imply trust-free or failure-free completion; fault tolerance and decentralization are therefore assessed separately. Ballot privacy hides choices, whereas identity-ballot unlinkability additionally depends on the credential and submission channel and is discussed in the text rather than scored in the table. The three-valued notation

records qualifications in the original threat models instead of treating every claimed feature as unconditional.

Early blockchain boardroom protocols emphasize self-tallying. McCorry et al. [11] adapt the Open Vote Network to Ethereum, but retain fairness, abort, and scale limitations. Seifelnasr et al. [30] move tally computation to an untrusted administrator and commit its trace in a Merkle tree: anyone can in principle recompute the tally, but practical completion relies on the administrator and at least one honest challenger. Li et al. [12] add recoverable commitments, and PriScore [13] extends this line to score ballots. Their recovery paths require cooperation from the remaining voters, so decentralization or fault recovery does not imply vote-and-go. BBB-Voting [20] supports robust 1-out-of- k tallying; its batching extension improves scale, while its recovery phase still requires voter participation. Voting booths further parallelize this design [19].

Threshold and PVSS systems move recovery work from voters to talliers. Schoenmakers [10] gives a universally verifiable binary construction whose tally needs threshold reconstruction rather than strict self-tallying. Bagheri et al.'s 3PVSS [36] packs candidate-wise binary choices and reduces multi-candidate verification to $O(m(n + \ell))$, but requires $n \geq 2t + \ell$, threshold reconstruction, and no voter anonymity. Huang et al. [14] also use Shamir recovery to tolerate absent voters. However, the election authority knows each ballot blind, and candidate keys are revealed at tally time, enabling it to open individual ballots; the published verification equation also leaves the z_y response unused. Ordinos [18] instead combines threshold Paillier encryption and MPC for plurality or tally-hiding result functions. It provides vote-and-go after ballot submission, but depends on authentication, administration, trusted voting devices, and a trustee threshold, and it is not a native score-ballot protocol.

Other schemes prioritize anonymity or receipt-freeness through additional roles. Miao et al. [24] combine traceable ring signatures with homomorphic time-lock puzzles for score voting, but their ballot privacy is explicitly time-bounded. Blind Vote [25] uses blind signatures and relays and accepts flexible plaintext vote structures; private scores would additionally need a score-validity proof, and administrator failure can cancel an election. ACB-Vote [35] proves ballot anonymity and related properties using convertible signatures, but voters must publish tally tags and two converters must not collude. ProvotumN [23] supports flexible ballots through a re-encryption mixnet, while its sealers, threshold decryption, voting authority, and trusted voting device make it neither self-tallying nor fully trust-free. Dimitriou et al. [33] obtain coercion resistance and universal verification using a randomizer token and zkSNARKs, introducing corresponding hardware and setup assumptions.

Our construction occupies a different point in this design space. It directly supports bounded, multi-option score vectors, public ballot and share verification, streaming aggregation, threshold completion, and vote-and-go. Its tally nevertheless requires a reconstruction quorum, and the RB-PVSS core does not by itself provide voter anonymity, which must come

Table II: Comparison of representative privacy-preserving voting schemes

Research	Cryptographic primitives	Pub. verif.	Integ.	Ballot privacy	Fault tol.	Scal.	Dec.	Vote-and-go	Multi. cand.	Score
McCorry et al. [11]	ECC+(0,1)ZKRP	✓	✓	✓	✗	✗	△	✓	✗	✗
Seifelnasr et al. [30]	Merkle tree	✓	✓	✓	✗	✓	△	✓	✗	✗
Li et al. [12]	ElGamal+(0,1)ZKRP	✓	✓	✓	✓	✗	△	✗	✗	✗
Yang et al. [13]	ElGamal+ZKRP	✓	✓	✓	△	✓	✗	✗	✓	✓
Homoliak et al. [20]	MPC	✓	✓	✓	✓	△	✓	✗	✓	✗
Huang et al. [14]	ElGamal+SS+ZKRP	△†	△†	△	✓	△	✗	✓	✓	✓
Küsters et al. [18]	Paillier+MPC	✓	✓	✓	✓	✓	△	✓	✓	✗
Killer et al. [23]	DKG+NIZK	✓	✓	✓	✓	✓	△	✓	✓	✗
Miao et al. [24]	Ring signature	✓	✓	△	△	✗	✗	✓	✓	✓
Goharshady et al. [25]	Blind signatures	✓	✓	✓	△	✓	✗	✗	✓	△
Xue et al. [35]	Convertible signature	✓	✓	✓	△	✓	✗	✗	✓	✓
Schoenmakers [10]	PVSS+(0,1)ZKRP	✓	✓	✓	✓	✗	✓	✓	✗	✗
Bagheri et al. [36]	3PVSS+NIZK	✓	✓	✓	✓	✓	✓	✓	✓	✗
Ours	PVSS+ZKRP	✓	✓	✓	✓	✓	✓	✓	✓	✓

Pub. verif.: public verifiability; Integ.: integrity; Fault tol.: fault tolerance; Scal.: scalability; Dec.: decentralization; Multi. cand.: multiple candidates. Here ✓ means that the property is proved under an explicit threat model or fully realized in the implementation; △ denotes conditional or partial support, including reliance on an extra trusted role or extension; and ✗ means that the property is explicitly unsupported. † Huang et al.’s published verification equations do not use response z_y , so correctness of the blinded first ElGamal component is not established as written; moreover, the election authority can recover the ballot blinds and later-opened candidate keys expose individual choices.

from an unlinkable credential and submission layer. These distinctions align the claims in this section with Table II.

VIII. EVALUATION AND IMPLEMENTATION

We evaluate three questions motivated by the deployment challenges in the Introduction: whether the complete RB-PVSS voting path is practical at representative DAO parameters, how its voter and tallying costs scale, and when direct range packing with streaming aggregation is preferable to prior score-voting designs.

A. Implementation and Evaluation Methodology

We implement all six RB-PVSS interfaces in Go and the public verification, streaming aggregation, and mask-reconstruction operations in Solidity. The off-chain implementation uses go-ethereum’s BN254 library; the Solidity 0.8.25 contract is compiled with optimization, via-IR, and the Paris EVM target. Experiments run on a 10-core Apple M4 MacBook Air with 16 GB RAM and macOS 26.5.2; chain measurements use an Ethereum network.

For the scalability experiment, we set $S = [0, 10]$, choose $n = 10, 20, \dots, 100$ and $l \in \{1, 10, 19\}$, retain the conservative family $l \leq n/3$, and set $t = \lfloor (n + l)/2 \rfloor$. Off-chain means use 20 measurements per non-setup operation and five for Setup, with one OS thread and garbage collection disabled. Transcript sizes use the exact fixed-width contract encoding. Transaction gas comes from successful receipts, and the on-chain group-operation portion of RB-PVSS.Recon is measured after t accepted shares. To isolate dependence on the accepted-voter count, we additionally submit ten consecutive ballots at $(n, l, t) \in \{(10, 5, 7), (50, 10, 30), (100, 19, 59)\}$. Figure 9 extends each measured second-ballot cost to $m = 1000$ as an analytical steady-state projection. Tests cover successful round trips, malformed proofs, replayed ballots, and Go–Solidity verification consistency. The cross-protocol experiment uses $(n, l, t) = (10, 5, 7)$.

B. Analytical Costs

Denote A_0, E_0, E_1 , and P as point addition in $\mathcal{G}_0$, scalar multiplication in $\mathcal{G}_0$, scalar multiplication in $\mathcal{G}_1$, and bilinear mapping, respectively. We additionally use PE for one polynomial evaluation. Hashes, finite-field arithmetic, public bounded decoding, and point additions internal to proof generation are omitted unless they dominate aggregation. For EVM serialization, the prover transmits $F_{jd}^{\text{base}} \in \mathcal{G}_0$ such that $F_{jd} = e(F_{jd}^{\text{base}}, g_1)$, and the Fiat–Shamir challenge binds this representation.

The counts combine the SCRAPE transcript, one range-binding proof per coordinate, and the DLEQ proofs used for decryption-share verification. Setup is reusable across compatible elections. A ballot’s generation, verification, and serialized size depend on n and l , but not on the accepted-ballot count m ; accepting it adds only $n + l$ aggregate-group operations. After the deadline, verification and reconstruction group operations depend on t and l .

Table III: Theoretical Computation and Output Sizes of Our Scheme

Phase	Entity	Operation	Comp.	Output size
SETUP	Setup parties	RB-PVSS.Setup	$(n + S)E_0 + E_1$	$ pp $
	SC	parameter storage	-	$ proposal + pp $
VOTING	Voter j	RB-PVSS.Share	$(9l + 4n)E_0 + lP$	$(6l + 4n) G_0 + (4l + n + 1) Z_g $
	SC for voter j	RB-PVSS.DVerify	$(5n + 9l)E_0 + 4lP$	-
		Aggregate	$(n + l)A_0$	-
TALLYING	Tallier i	RB-PVSS.Decrypt	$3E_0$	$3 G_0 + 2 Z_g $
	SC	RB-PVSS.PVerify; RB-PVSS.Recon	$(4t + tl)E_0 + tlA_0$	$l G_0 $

The output-size column reports data produced by the listed entity, except that the Setup row reports persistent parameter storage; inputs already counted at their sender are not counted again. Our scheme requires $t - f \geq l$ and $n - f \geq t$; choosing $t = f + l$ gives $n \geq 2f + l$. Dealer verification is mandatory before a ballot affects the aggregate.

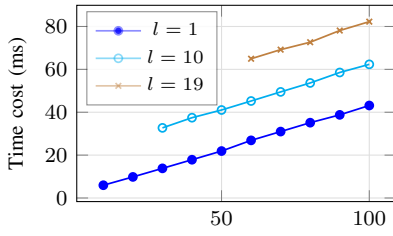


Figure 6: The time cost of RB-PVSS.Share vs. n

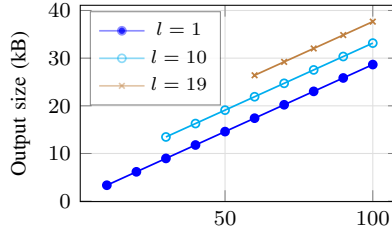


Figure 7: Serialized RB-PVSS.Share output size vs. n

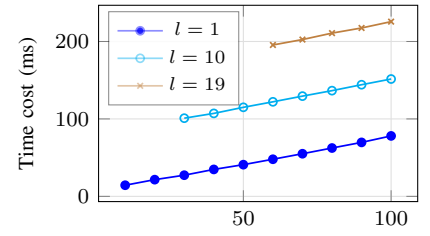


Figure 8: The time cost of RB-PVSS.DVerify vs. n

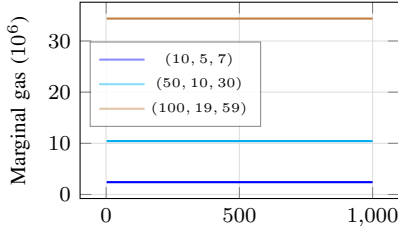


Figure 9: Gas of a ballot vs. m ; each line with a different (n, l, t)

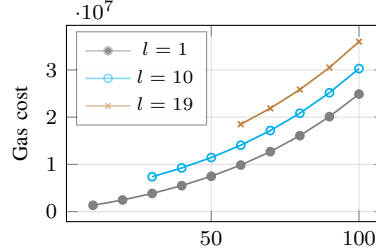


Figure 10: Gas for one accepted ballot vs. n

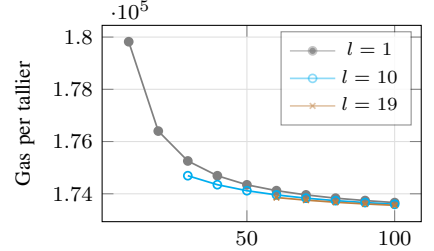
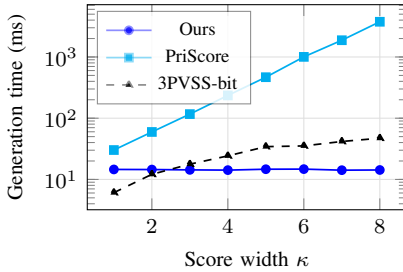
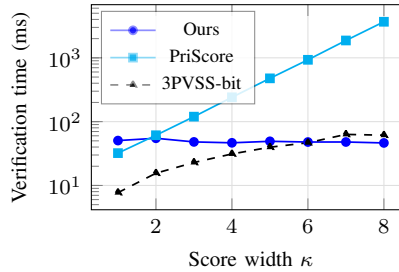


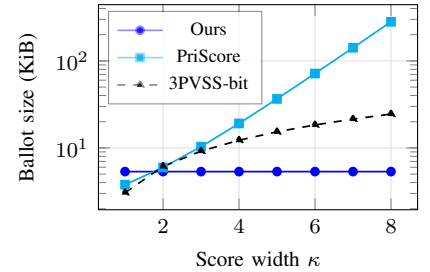
Figure 11: Per-tallier decryption-share gas vs. n



(a) Ballot generation



(b) One-ballot verification



(c) Serialized public transcript

Figure 12: Online cost of supporting scores in $[0, 2^\kappa)$, with $n = 10$, $l = 5$, $t = 7$, and $B = 2^\kappa - 1$.

C. Performance and On-Chain Scaling

Figures 6–11 evaluate the implementation. The first row reports off-chain ballot work and communication; the second reports steady-state marginal ballot gas versus the accepted-voter count m , one accepted-ballot transaction versus n , and one tallier submission.

The off-chain curves grow smoothly with n and l . At the largest tested configuration $(n, l, t) = (100, 19, 59)$, ballot generation remains below 0.1 s, public verification below 0.25 s, and the serialized transcript below 40 KiB. This demonstrates that the complete RB-PVSS transcript, including the F^{base} encoding, is practical on commodity hardware.

Figure 10 measures one successful accepted-ballot transaction at each fixed (n, l) , not cumulative gas over m . Figure 11 is nearly independent of l because each tallier proves one aggregate decryption; its largest-point mean is about 0.174 million gas. Figure 9 isolates voter-count scaling. For $(n, l, t) = (10, 5, 7), (50, 10, 30), (100, 19, 59)$, the measured gas of the second accepted ballot is respectively 2.405107, 10.432492, and 34.377800 million. Across ballots 2–10, the

peak-to-peak variation is at most 0.003% of the corresponding mean. The horizontal extensions to $m = 1000$ reflect that each accepted ballot verifies one fixed-size transcript and updates the same $n + l$ aggregate elements, independently of prior ballots. This establishes constant marginal voter cost with respect to m ; total voting-phase gas still grows linearly with the number of accepted ballots and remains subject to blockchain throughput.

Contract deployment is a reusable Setup cost of about 10.1 million gas in these configurations. At the fixed gas-price and ETH/USD snapshot used in the experiment, one voter transaction at $n = 100$ costs about \$6.8–\$9.9 *per voter*, while one decryption-share submission costs about \$0.048 *per tallier*.¹

D. Comparative Evaluation

The comparison follows the challenges identified in the Introduction. Table II compares protocol mechanisms and

¹Illustrative conversion using 0.114 Gwei and 1 ETH = \$2,409.50, observed September 3, 2026; the values scale linearly with market prices.

Table IV: Computation Costs of Multi-Option Score-Voting Protocols

Scheme	Vote per voter	Verify m ballots	Streaming aggregation	Post-deadline finalization
PriScore [13]	$(14B + 17)lE_0 + O(E_0)$	$m(17B + 8)E_0$	Deferred	$0E_0$
3PVSS-bit adaptation [36]	$\kappa[2(n + l)(E_0 + PE) + 6lE_0]$	$m\kappa[(n + l)(2E_0 + PE + A_0) + 8lE_0]$	No; deferred to tally	$\kappa[5(f + l)E_0 + (f + m)lA_0]$
Ours	$(9l + 4n)E_0 + lP$	$m[(5n + 9l)E_0 + 4lP]$	$(n + l)A_0$	$(7t + tl)E_0 + tlA_0$

Here $m = |\mathcal{A}|$ is the number of accepted ballots in the all-valid comparison, n is the number of talliers, t is the reconstruction threshold, and l is the number of ballot coordinates (governance options); B is PriScore’s fixed total-score budget. The symbols A_0 , E_0 , E_1 , and P denote point addition in $\mathcal{G}_0$, scalar multiplication in $\mathcal{G}_0$, scalar multiplication in $\mathcal{G}_1$, and bilinear mapping, respectively. Counts across different groups are not interpreted as identical wall-clock costs. Hashes and finite-field operations are excluded; polynomial evaluations are shown only where the 3PVSS source counts them explicitly. Public bounded decoding is excluded uniformly from all rows. *Deferred* means that the scheme does not maintain the corresponding aggregate as each ballot is accepted. PriScore’s $0E_0$ records no additional exponentiation in the cited post-deadline accounting; it does not mean that its m -dependent public transcripts occupy no storage or need not be read by a finalizer. For the 3PVSS row, the reconstruction threshold is $f + l$, $n \geq 2f + l$, and $b - a + 1 = 2^\kappa$. This row is a black-box score adaptation obtained by running κ independent instances of the paper’s native binary-vector protocol; it is not a score-voting construction claimed by 3PVSS. Its cited tally procedure first accumulates accepted ballots at cost κmlA_0 , which is included in the final column, then reconstructs κ bit planes. Its per-voter communication is $\kappa[(n + 5l)|\mathcal{G}_0| + (f + 5l + 1)|\mathcal{Z}_q|]$, compared with $(4n + 6l)|\mathcal{G}_0| + (n + 4l + 1)|\mathcal{Z}_q|$ for one ballot in our scheme. Vote includes all mandatory ballot-generation operations; hence, the PriScore row includes Commit and Vote. In our scheme, ciphertext and masked-score-commitment aggregation is performed incrementally whenever a ballot is accepted. Our reusable setup costs $(n + b - a + 1)E_0 + E_1$ and stores $O(b - a)$ range parameters; this one-time cost is reported separately from per-ballot Vote.

failure paths, whereas Table IV isolates the score-voting costs of PriScore, the explicitly labeled 3PVSS-bit adaptation, and our scheme. Executable measurements of these three schemes use the same Go BN254 backend; Huang et al. appears only as a separately implemented tally-path baseline. ACB-Vote is retained only in the qualitative literature comparison. Cross-protocol timings use one warm-up and ten measured repetitions with garbage collection disabled, and all implemented round trips recover the expected totals and reject malformed inputs.

Figure 12 confirms the predicted online trends. The generation time, verification time, and transcript size of our scheme remain nearly constant as κ grows; PriScore grows with $B = 2^\kappa - 1$, while the 3PVSS-bit adapter grows approximately linearly with its κ binary instances. Native binary 3PVSS is still lighter at small κ , and our reusable $O(2^\kappa)$ setup is excluded from these per-ballot measurements. The advantage is therefore direct support for moderate nonbinary domains after setup reuse, not universal superiority.

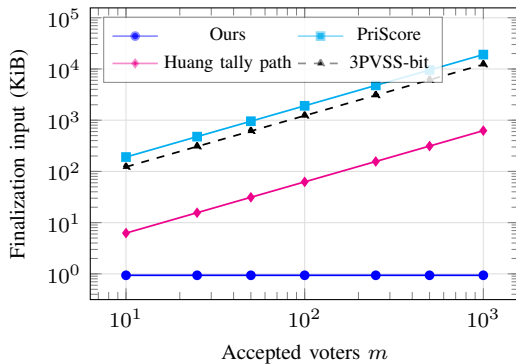


Figure 13: Cryptographic working set processed at finalization, for $n = 10$, $l = 5$, $t = 7$, and the common score domain $[0, 15]$, where $B = 15$ and $\kappa = 4$.

Figure 13 examines the $\kappa = 4$ point. Our scheme retains a sub-KiB $n + l$ -element active aggregate throughout, whereas the other finalization paths grow linearly with m , reaching

from hundreds of KiB to tens of MiB at $m = 1000$. This metric is the fixed-width cryptographic input processed by finalization, not total immutable chain history or wall-clock time. PriScore’s $0E_0$ in Table IV therefore does not imply zero transcript input, and Huang remains only a separately labeled tally-path baseline.

Together, the two experiments identify the regime in which our scheme is advantageous rather than supporting a universal dominance claim. When the score range is moderate, setup is reused, and $m \gg t$, direct packing avoids repeated online work and streaming aggregation bounds the active finalization state. The corresponding measured costs are pairing-heavy ballot verification, an $O(2^\kappa)$ reusable setup, and on-chain reconstruction.

IX. CONCLUSION

We presented RB-PVSS and a publicly verifiable threshold score-voting system built from SCRAPE PVSS and range-binding proofs. A voter directly packs an independent-range score vector, the contract verifies and aggregates each accepted ballot atomically, and any t valid tallier shares enable public reconstruction without further voter participation.

The evaluation shows practical ballot generation and verification on commodity hardware, a per-ballot cost independent of the accepted-ballot count, and an active finalization state independent of m . Relative to PriScore and the explicit 3PVSS-bit adaptation, the main advantage appears for moderate score domains, reusable setup, and $m \gg t$: direct range packing avoids budget- or bit-decomposition-dependent online work, while streaming aggregation avoids retaining every ballot as active tally input. This design point incurs pairing-heavy verification, range-dependent setup, and on-chain reconstruction cost. The current construction targets additive totals and means; joint ballot constraints and non-additive tally rules require additional proof relations or tally logic.

REFERENCES

- [1] Wang, S., Ding, W., Li, J., Yuan, Y., Ouyang, L. and Wang, F. Y. Decentralized autonomous organizations: Concept, model, and applications.

IEEE Transactions on Computational Social Systems, 2019, 6(5), 870-878.

- [2] Sharma, T., Potter, Y., Pongmala, K., Wang, H., Miller, A., Song, D. and Wang, Y. Unpacking how decentralized autonomous organizations (daos) work in practice. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2024, pp. 416-424.
- [3] Ethereum, What is a DAO? <https://ethereum.org/en/dao>, Accessed at: 2025-02-25.
- [4] Zhang, J., Zhang, B., Nastenkov, A., Balogun, H. and Oliynykov, R. Privacy-preserving decision-making over blockchain. *IEEE Transactions on Dependable and Secure Computing*, 2022, 20(6), pp.4648-4663.
- [5] Qin, R., Ding, W., Li, J., Guan, S., Wang, G., Ren, Y. and Qu, Z. Web3-based decentralized autonomous organizations and operations: Architectures, models, and mechanisms. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2022, 53(4), pp.2073-2082.
- [6] Ding, Q., Liebau, D., Wang, Z. and Xu, W. A survey on decentralized autonomous organizations (DAOs) and their governance. *World Scientific Annual Review of Fintech*, 2023, 2350001.
- [7] Gilson, C., Bouraga, S. Enhancing the democratic nature of voting processes within decentralized autonomous organizations. *Digital Policy, Regulation and Governance*, 2024, 26(2), 169-187.
- [8] Tan, J. Z., Merk, T., Hubbard, S., Oak, E. R., Pirovich, J., Rennie, E., Boneh, D. Open Problems in DAOs. *arXiv preprint arXiv:2310.19201*, 2023.
- [9] Fan, Y., Zhang, L., Wang, R., Imran, M. A. Insight into Voting in DAOs: Conceptual Analysis and A Proposal for Evaluation Framework. *IEEE Network*, 2024, 38(3), 92-99.
- [10] Schoenmakers, B. A simple publicly verifiable secret sharing scheme and its application to electronic voting. In *CRYPTO*, 1999, pp. 148-164.
- [11] McCorry, P., Shahandashti, S. F., Hao, F. A smart contract for boardroom voting with maximum voter privacy. In *Financial Cryptography and Data Security*, 2017, pp. 357-375.
- [12] Li, Y., Susilo, W., Yang, G., Yu, Y., Liu, D., Du, X., Guizani, M. A blockchain-based self-tallying voting protocol in decentralized IoT. *IEEE Transactions on Dependable and Secure Computing*, 2020, 19(1), 119-130.
- [13] Yang, Y., Guan, Z., Wan, Z., Weng, J., Pang, H. H., Deng, R. H. Priscore: blockchain-based self-tallying election system supporting score voting. *IEEE Transactions on Information Forensics and Security*, 2021, 16, 4705-4720.
- [14] Huang, J., He, D., Chen, Y., Khan, M. K., and Luo, M. A Blockchain-Based Self-Tallying Voting Protocol With Maximum Voter Privacy. *IEEE Transactions on Network Science and Engineering*, 2022, 9(5), 3808-3820.
- [15] Snapshot, <https://docs.snapshot.box/proposals/voting-types>. Accessed at: 2025-02-25
- [16] Zhang, F., and Kim, K. ID-based blind signature and ring signature from pairings. In *ASIACRYPT*, 2002, pp. 533-547.
- [17] Li, H., Kankanala, A. R., and Zou, X. A taxonomy and comparison of remote voting schemes. In *the 23rd International Conference on Computer Communication and Networks*, 2014, pp. 1-8.
- [18] Küsters, R., Liedtke, J., Müller, J., Rausch, D., and Vogt, A. Ordinos: a verifiable tally-hiding remote e-voting system. *IEEE European Symposium on Security and Privacy*, 2020, pp. 216-235.
- [19] Stančíková, I. and Homoliak, I. Sbvote: Scalable self-tallying blockchain-based voting. In *the 38th ACM/SIGAPP symposium on applied computing*, 2023, pp. 203-211.
- [20] Homoliak, I., Li, Z., Szalachowski, P. BBB-Voting: Self-Tallying End-to-End Verifiable 1-out-of-k Blockchain-Based Boardroom Voting. In *IEEE International Conference on Blockchain*, 2023, pp. 297-306.
- [21] Khader, D., Smyth, B., Ryan, P., and Hao, F. A fair and robust voting system by broadcast. *Lecture Notes in Informatics*, 2012, 285-299.
- [22] Parno, B., Howell, J., Gentry, C., and Raykova, M. Pinocchio: Nearly practical verifiable computation. *Communications of the ACM*, 2016, 59(2), 103-112.
- [23] Killer, C., Eck, M., Rodrigues, B., von der Assen, J., Staubli, R., Stiller, B. ProvotumN: Decentralized, Mix-Net-based, and Receipt-free Voting System. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2022, pp. 1-9.
- [24] Miao, M., Tang, L., Li, J., and Zhang, X. New Blockchain-based Publicly Traceable Self-tallying Voting Protocol. *IEEE Transactions on Network Science and Engineering*, 2024, 11(2), 2034-2046.
- [25] Goharshady, A. K., and Lin, Z. Blind vote: Economical and secret blockchain-based voting. In *IEEE International Conference on Blockchain*, 2024, pp. 46-53.
- [26] Wood, G. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 2014.
- [27] Reed, I. S., Solomon, G. Polynomial codes over certain finite fields. *Journal of the society for industrial and applied mathematics*, 1960, 8(2), 300-304.
- [28] Alsadi, M., Casey, M., Dragan, C.C., Dupressoir, F., Riley, L., Sallal, M., Schneider, S., Treharne, H., Wadsworth, J. and Wright, P. Towards End-to-End Verifiable Online Voting: Adding Verifiability to Established Voting Systems. *IEEE transactions on dependable and secure computing*, 2024, 21, 3357-3374.
- [29] Cascudo, I., David, B. SCRAPE: Scalable randomness attested by public entities. In *Applied Cryptography and Network Security*, 2017, pp. 537-556.
- [30] Seifelnasr, M., Galal, H. S., Youssef, A. M. Scalable open-vote network on ethereum. In *Financial Cryptography and Data Security*, 2020, pp. 436-450.
- [31] Hao, F., Ryan, P.Y. and Zieliński, P. Anonymous voting by two-round public discussion. *IET Information Security*, 2010, 4(2), 62-67.
- [32] Li, M., Xue, K., Luo, X., Sun, W., Wei, D.S., Sun, Q. and Lu, J. S³ Voting: A Blockchain Sharding Based E-voting Approach with Security and Scalability. *IEEE Transactions on Dependable and Secure Computing*, 2024, Accepted.
- [33] Dimitriou, T. Efficient, coercion-free and universally verifiable blockchain-based voting. *Computer Networks*, 2020, 174, 107234.
- [34] Sampigethaya, K., Poovendran, R. A framework and taxonomy for comparison of electronic voting schemes. *computers & security*, 2006, 25(2), 137-153.
- [35] Xue, W., Yang, Y., Li, Y., Pang, H.H. and Deng, R.H. ACB-vote: Efficient, flexible, and privacy-preserving blockchain-based score voting with anonymously convertible ballots. *IEEE Transactions on Information Forensics and Security*, 2023, 18, 3720-3734.
- [36] Bagheri, K., Makri, E., and Suryakari, D. K. Packed pre-constructed PVSS for randomness generation and e-voting. In *Security and Cryptography for Networks (SCN)*, 2026; full version: Cryptology ePrint Archive, Paper 2026/1119.
- [37] Damgård, I. On Σ -protocols. *Lecture Notes, University of Aarhus, Department for Computer Science*, 2002.
- [38] Basin, D., Radomirović, S., Schmid, L. Dispute resolution in voting. In *IEEE 33rd Computer Security Foundations Symposium*, 2020, pp. 1-16.
- [39] Camenisch, J., Chaabouni, R., Shelat, A. Efficient protocols for set membership and range proofs. In *ASIACRYPT*, 2008, pp. 234-252.
- [40] Boneh, D., Boyen, X. Short signatures without random oracles. In *EUROCRYPT*, 2004, pp. 56-73.



Liang Zhang was born in 1989. He received a bachelor's degree in computer science from Huazhong University of Science and Technology, Wuhan, in 2012. After five years of being an engineer in the IT industry, he joined Fudan University in 2017 and acquired his Ph.D. degree in 2022. He became an associate professor at Hainan University and he is currently a research assistant professor in The Hong Kong University of Science and Technology. His research interests include Web 3.0, blockchain and applied cryptography. His recent publications are included in *IEEE TC*, *IEEE TIFS*, *IEEE TMC*, *IEEE TSC*, *IEEE TBD*, *IEEE TCC*, *CJ*, *CSCWD'25*, *APWEB'23*, *CF'22* and *SACMAT'22*.



Xingyu Wu was born in 1998. She received a bachelor's degree in information security from Southwest University of Science and Technology, Mianyang, in 2020. After two years, she joined Hainan University in September 2022. Now, she is pursuing her Ph.D. degree in the School of Cyberspace Security (School of Cryptology) at Hainan University. Her research interests consist of blockchain and CP-ABE. Her recent research was published in BIBM'23.



Haibin Kan was born in 1971. He received a Ph.D. from Fudan University, Shanghai, China, 1999. From June 2002 to February 2006, he was with the Japan Advanced Institute of Science and Technology as an assistant professor. He went back Fudan University in February 2006, where he is currently a full professor. He is also the Director of the Shanghai Blockchain Engineering Research Center. His research interests include coding theory, cryptography, and computation complexity.



Jiheng Zhang received both Ph.D. and M.S. degrees in operations research from Georgia Institute of Technology, MS degree in mathematics from the Ohio State University, and BS degree in mathematics in Nanjing University. He joined Industrial Engineering and Decision Analytics (IEDA) of the Hong Kong University of Science and Technology (HKUST) as an Assistant Professor in 2009, and was promoted to Associate Professor in 2015 and then to full Professor in 2020. He now is the department head of IEDA of HKUST.